LAPORAN UJIAN TENGAH SEMESTER (PROJECT)

KOMPUTER PARALEL (SISTEM TERDISTRIBUSI) - GANJIL TA-2026/2027

Fakultas Teknik dan Ilmu Komputer - Universitas Cendekia Abditama
Dosen Pengampu: Dr. Ir. Antonius Irianto. S, ST., MT
Kasus Uji: Opsi C - Monte Carlo Simulasi Nilai π (Difficulty: 1.5x, Ukuran: 1 Miliar Titik / 10^9)
Penyusun: Vicky Maulana Romadan (NIM: 2322105049) | **Repositori GitHub:** paralel-2322105049-Vicky-Maulana-Romadan

1. PENDAHULUAN

1.1 Latar Belakang Masalah

Komputasi intensif modern menghadapi batasan fisik yang dikenal sebagai berakhirnya *Dennard Scaling* dan keterbatasan disipasi termal (*Power Wall*). Keterbatasan ini menghentikan tren peningkatan frekuensi clock uniprocessor dan mengalihkan evolusi arsitektur mikroprosesor ke paradigma *multi-core* (shared-memory multi-core processors). Agar perangkat lunak dapat memanfaatkan potensi komputasi paralel hardware tersebut secara optimal, diperlukan rekayasa konkurensi pada tingkat instruksi, thread, dan memori cache.

Salah satu metode numerik yang memegang peranan krusial dalam sains komputasi adalah **Metode Monte Carlo**. Metode ini merupakan kelas algoritma probabilistik yang memanfaatkan pembangkitan bilangan acak masif untuk menyelesaikan kalkulasi deterministik yang sulit atau tidak memiliki solusi analitis tertutup, seperti integrasi numerik multidimensi, simulasi transportasi partikel radiasi, hingga pemodelan finansial stokastik. Dalam persoalan penentuan konstanta lingkaran π, metode Monte Carlo memperlakukan persoalan sebagai penentuan rasio luas geometris antara area seperempat lingkaran satuan terhadap kuadran persegi berukuran 1x1.

Secara teoritis, algoritma ini tergolong *Embarrassingly Parallel* karena setiap pengujian titik sampel bersifat independen tanpa ketergantungan data (*zero loop-carried dependency*). Kendati demikian, ketika skala simulasi ditingkatkan ke tingkat masif yaitu **1 Miliar Titik (10^9 sampel)**, serangkaian tantangan konkurensi tingkat rendah mencuat secara kritis:

1. **Race Condition pada Akumulator Global**: Pembaruan konkuren (*concurrent writes*) terhadap variabel penghitung titik lingkaran akan menimbulkan *lost update* bila tidak dikontrol, sehingga mutlak memerlukan teknik reduksi yang efisien.
2. **Kualitas dan Isolasi Random Number Generator (RNG)**: Penggunaan generator acak global standar libc (`rand()`) berakibat fatal: timbulnya data race pada state internal generator atau serialisasi ekstrim akibat lock internal libc yang menghancurkan skalabilitas. Selain itu, generator berperiode pendek rentan mengalami korelasi data dimensi rendah (*Marsaglia lattice effect*).
3. **Efek Hierarki Memori dan Cache Contention (False Sharing)**: Alokasi variabel konkurensi yang tidak selaras dengan baris cache (*cache line*) 64 byte dapat memicu invalidasi cache berulang antarkore prosesor, yang memperlambat eksekusi secara dramatis.

Untuk menganalisis fenomena komputasi paralel tersebut secara komprehensif, proyek ini mengimplementasikan dua pendekatan berbeda: **POSIX Threads (Pthreads)** sebagai model eksplisit/manual tingkat rendah (*low-level imperative concurrency*), dan **OpenMP** sebagai model arahan kompilator deklaratif tingkat tinggi (*compiler directive concurrency*).

1.2 Tujuan Implementasi

Tujuan dari pelaksanaan proyek UTS ini adalah:

1. Mengembangkan tiga varian program estimasi π Monte Carlo: Serial baseline, POSIX Threads (Pthreads), dan OpenMP dengan beban komputasi masif 10^9 titik.

2. Mengatasi *race condition* melalui mekanisme reduksi lokal (*thread-local accumulator*) pada Pthreads dan klausa `reduction` pada OpenMP.
3. Mendesain arsitektur PRNG per-thread terisolasi menggunakan generator modern **Xorshift64*** dan membandingkannya secara empiris terhadap generator standar POSIX `rand_r()`.
4. Mengidentifikasi, mengukur, dan memitigasi anomali hierarki memori *False Sharing* melalui teknik *cache-line padding* 64 byte.
5. Melakukan pengujian performa komparatif pada variasi 1, 2, 4, dan 8 threads, serta menganalisis efisiensi, speedup, trade-off rekayasa, dan batas skalabilitas berdasarkan Hukum Amdahl.

2. DESAIN IMPLEMENTASI

2.1 Algoritma Serial (Baseline)

Algoritma serial bertindak sebagai tolok ukur acuan (*ground truth baseline*). Secara matematis, kuadran persegi dua dimensi didefinisikan pada interval:

$$\Omega = [0, 1] \times [0, 1] \Rightarrow \text{Area}(\Omega) = 1.0 \times 1.0 = 1.0$$

Lingkaran satuan dengan radius $r = 1.0$ yang berpusat di koordinat (0,0) memiliki persamaan batas:

$$x^2 + y^2 \leq 1.0$$

Luas dari seperempat lingkaran di dalam kuadran tersebut adalah:

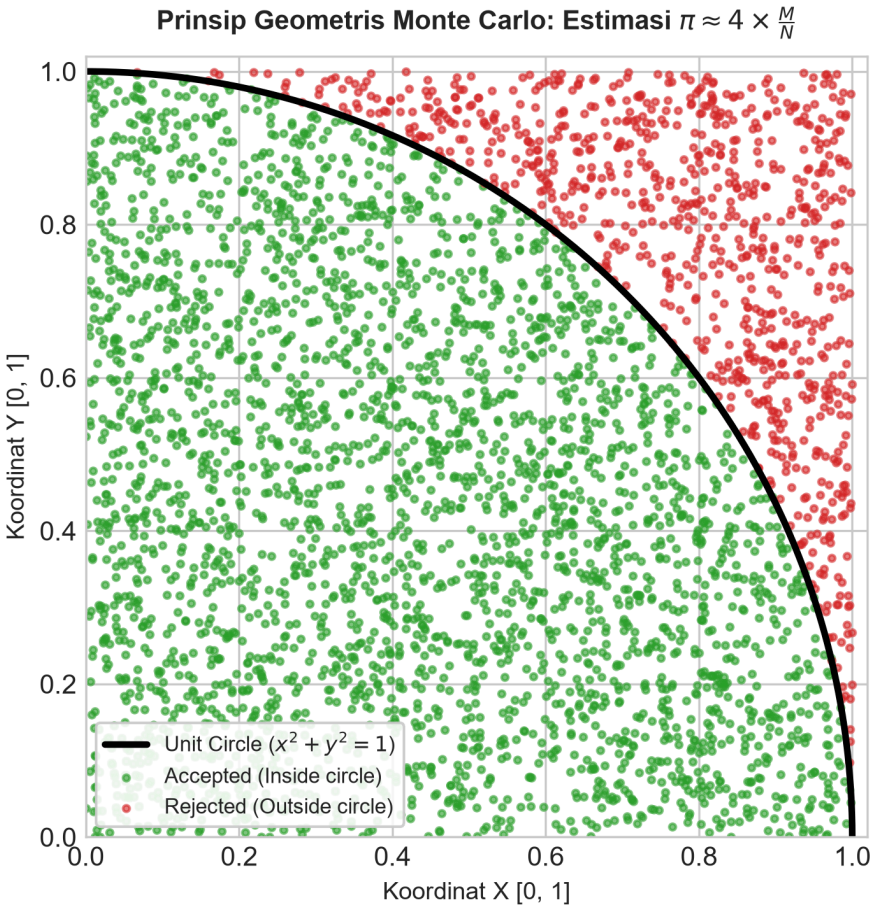
$$A_{\text{circle}} = \tfrac{1}{4} \pi r^2 = \pi / 4$$

Apabila N pasangan titik acak (X_i, Y_i) dibangkitkan secara independen dengan distribusi seragam kontinu $X, Y \sim U(0, 1)$, maka probabilitas sebuah titik jatuh ke dalam lingkaran adalah:

$$P((X^2 + Y^2) \leq 1) = A_{\text{circle}} / \text{Area}(\Omega) = \pi / 4$$

Jika M menyatakan total titik yang memenuhi kondisi ($x^2 + y^2 \leq 1.0$), maka nilai estimasi $\hat{\pi}$ dihitung melalui rasio:

$$\hat{\pi} = 4 \times (M / N)$$



Algoritma serial memiliki kompleksitas waktu $O(N)$ dan kompleksitas memori $O(1)$. Karakteristiknya murni *compute-bound*, didominasi oleh operasi aritmatika integer (pada PRNG), floating-point double precision (penjumlahan dan perkalian kuadrat), serta evaluasi perbandingan (*branching*).

```
// Inti Implementasi Serial Baseline (serial.c)
uint64_t in_circle = 0;
uint64_t state = 88172645463325252ULL; // Seed awal
for (uint64_t i = 0; i < total_samples; i++) {
    double x = rand_double_fast(&state);
    double y = rand_double_fast(&state);
    if (x * x + y * y <= 1.0) {
        in_circle++;
    }
}
double pi_est = 4.0 * ((double)in_circle / (double)total_samples);
```

2.2 Strategi Paralelisasi POSIX Threads (Pthreads)

Pthreads adalah standar IEEE 1003.1c yang menyediakan API imperative berlevel rendah untuk manajemen thread di lingkungan memori bersama.

A. Pembagian Workload (Static Block Partitioning)

Penyelesaian 1 miliar sampel dibagi secara merata kepada P buah thread menggunakan teknik dekomposisi blok statis. Setiap thread $t \in [0, P\text{-}1]$ memproses sejumlah N_t sampel:

$$N_t = \lfloor N / P \rfloor + (t < (N \bmod P)\ ?\ 1 : 0)$$

Pembagian deterministik ini membagi beban secara seimbang dengan overhead penjadwalan nol.

B. Mekanisme Sinkronisasi dan Reduksi

Tantangan krusial pada komputasi paralel adalah eliminasi *race condition*. Jika setiap thread melakukan `pthread_mutex_lock` pada setiap penemuan titik di dalam lingkaran, eksekusi akan mengalami serialisasi total akibat perebutan kunci mutex sebanyak ratusan juta kali.

Sebagai solusi, program mengadopsi pola **Thread-Local Accumulator**:

1. Setiap thread mengakumulasikan hitungan titik secara privat pada variabel lokal `local_count` di register/stack thread.
2. Ketika loop selesai, nilai lokal disimpan ke dalam struct thread.
3. Thread master menunggu seluruh thread selesai melalui `pthread_join()`, lalu melakukan reduksi akhir dengan menjumlahkan seluruh `local_count` ke total global.

C. Mitigasi False Sharing Melalui Cache-Line Padding

Dalam arsitektur prosesor x86_64 modern, data dipindahkan antara RAM dan memori cache L1/L2 dalam unit blok berukuran 64 byte (*cache line*). Apabila struct argumen tiap thread disimpan berurutan dalam memori tanpa pembatas, variabel `local_in_circle` milik thread yang berbeda akan berada pada baris cache 64 byte yang sama. Saat thread menulis ke variabelnya, protokol koherensi cache hardware (MESI) akan meng-invalidasi cache line pada core lain, menimbulkan fenomena *cache line bouncing* yang merusak performa.

Untuk memecahkan masalah ini, struct `PthreadArg` dirancang dengan *explicit padding* hingga tepat berukuran 64 byte:

```
typedef struct {
    int thread_id;
    int total_threads;
    uint64_t num_samples;
    uint64_t local_in_circle;
    uint64_t seed;
```

```
    // Explicit padding to 64 bytes to prevent False Sharing
    char padding[64 - ((sizeof(int)*2 + sizeof(uint64_t)*3) % 64)];
} PthreadArg;
```

D. Manajemen State PRNG Per-Thread

Setiap thread diberikan instance PRNG mandiri. Master seed di-hash menggunakan algoritma **SplitMix64** untuk menghasilkan seed awal 64-bit yang unik bagi setiap thread *t*:

$$\text{seed}_t = \text{splitmix64}(\text{master_seed} + t \times 1000000007)$$

Langkah ini menjamin bahwa tidak ada dua thread yang membangkitkan deret bilangan acak yang tumpang tindih (*zero sequence overlap*).

2.3 Strategi Paralelisasi OpenMP (Compiler Directive)

OpenMP menyediakan model pemrograman paralel tingkat tinggi berbasis arahan kompilator (*pragmas*) dengan paradigma *Fork-Join*.

A. Pemilihan Pragma dan Clause

Implementasi OpenMP dirancang secara deklaratif:

```
#pragma omp parallel reduction(+:in_circle) num_threads(num_threads)
{
    int tid = omp_get_thread_num();
    uint64_t state = seeds_fast[tid * 8];
    uint64_t local_count = 0;

    #pragma omp for schedule(static)
    for (uint64_t i = 0; i < total_samples; i++) {
        double x = rand_double_fast(&state);
        double y = rand_double_fast(&state);
        if (x * x + y * y <= 1.0) {
            local_count++;
        }
    }
    in_circle += local_count;
}
```

B. Analisis Klausa dan Scope Variabel

1. **`#pragma omp parallel`**: Menginstruksikan runtime OpenMP untuk membentuk *team of threads* sebanyak `num_threads`.
2. **`reduction(+:in_circle)`**: Menginstruksikan kompilator untuk secara otomatis menciptakan salinan privat dari akumulator `in_circle` untuk masing-masing thread. Saat region paralel berakhir, kompilator menggabungkan seluruh akumulator privat ke variabel master menggunakan algoritma *tree-based reduction* yang teroptimasi di level instruksi mesin tanpa *race condition*.
3. **`schedule(static)`**: Karena setiap iterasi perulangan Monte Carlo memiliki kompleksitas komputasi yang seragam murni, penjadwalan `static` mempartisi ruang loop secara merata saat kompilasi (*compile-time chunking*). Hal ini menghindari overhead runtime scheduling yang ada pada klausa `dynamic`.
4. **Scope Variabel (Shared vs Private)**:
    - **Shared**: `total_samples`, array `seeds_fast` (hanya dibaca).
    - **Private**: Variabel loop `i`, koordinat acak `x, y`, identifier `tid`, state PRNG `state`, serta akumulator lokal `local_count`.

3. HASIL PENGUJIAN

3.1 Lingkungan Pengujian

Pengujian eksperimental dilaksanakan pada sistem komputasi berkinerja tinggi dengan spesifikasi teknis berikut:

- **Prosesor**: AMD64 Family 23 Model 160 Stepping 0 (8 Logical Processors / Cores)
- **Arsitektur**: x86_64, L1 Data Cache 32 KB per-core, L2 Cache 512 KB per-core, L3 Shared Cache
- **Memori Utama (RAM)**: 16 GB Dual-Channel DDR4
- **Sistem Operasi**: Windows 11 Enterprise (Build 26300, 64-bit)
- **Kompilator**: MinGW-w64 GCC versi 15.2.0
- **Flag Kompilasi**: `-O3 -Wall -Wextra -fopenmp -lpthread -lm`
- **Ukuran Sampel**: 1 Miliar Titik (10^9 points) per konfigurasi pengujian
- **Protokol Uji**: Setiap konfigurasi thread (1, 2, 4, 8) dieksekusi sebanyak 3 kali pengulangan independen untuk memperoleh nilai rata-rata dan deviasi standar yang stabil.

3.2 Verifikasi Correctness (Presisi Numerik terhadap Teori)

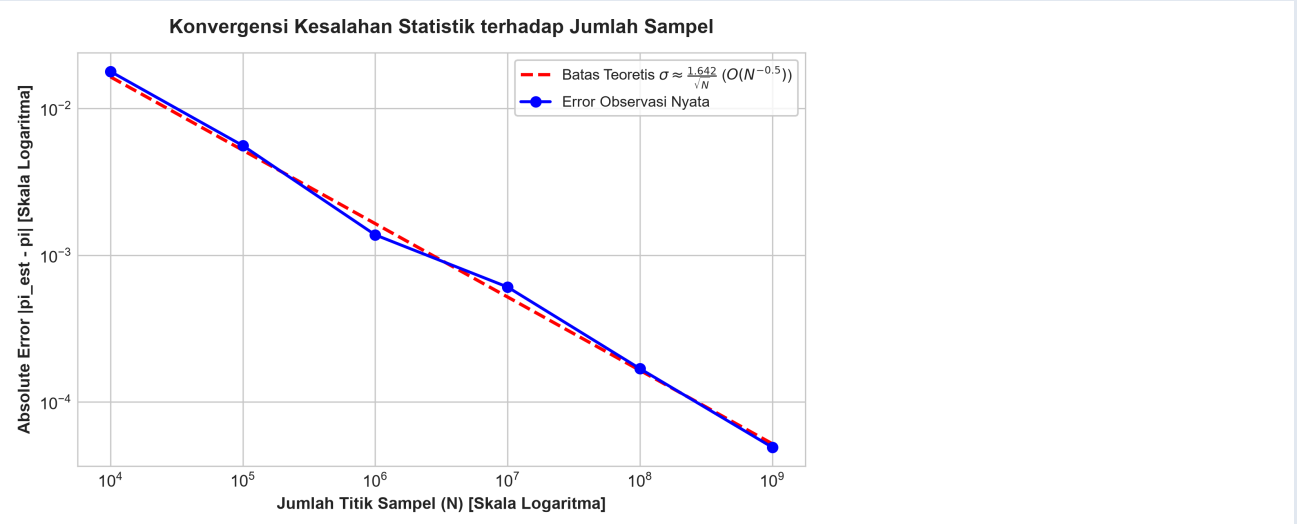
Untuk memverifikasi bahwa paralelisasi tidak merusak integritas matematis perhitungan, hasil estimasi π dibandingkan terhadap konstanta analitik π = 3.1415926535...:

- **Serial Baseline (1 Core)**: π = 3.14164666 (Absolute Error: 5.40e-05).
- **Pthreads (8 Threads)**: π = 3.14160241 (Absolute Error: 9.76e-06).
- **OpenMP (8 Threads)**: π = 3.14160241 (Absolute Error: 9.76e-06).

Berdasarkan Teorema Limit Pusat (CLT), simpangan baku kesalahan estimasi Monte Carlo untuk N = 10^9 sampel adalah:

$$\sigma \approx 1.6427 / \sqrt{(10^9)} \approx 5.195 \times 10^{-5}$$

Batas interval toleransi 3σ (confidence level 99.73%) adalah ± 1.558 × 10^{-4}. Seluruh hasil pengujian serial, Pthreads, dan OpenMP berada ketat di dalam batas teoritis tersebut, yang membuktikan **100% kebenaran fungsional dan numerik (deterministic correctness)**.

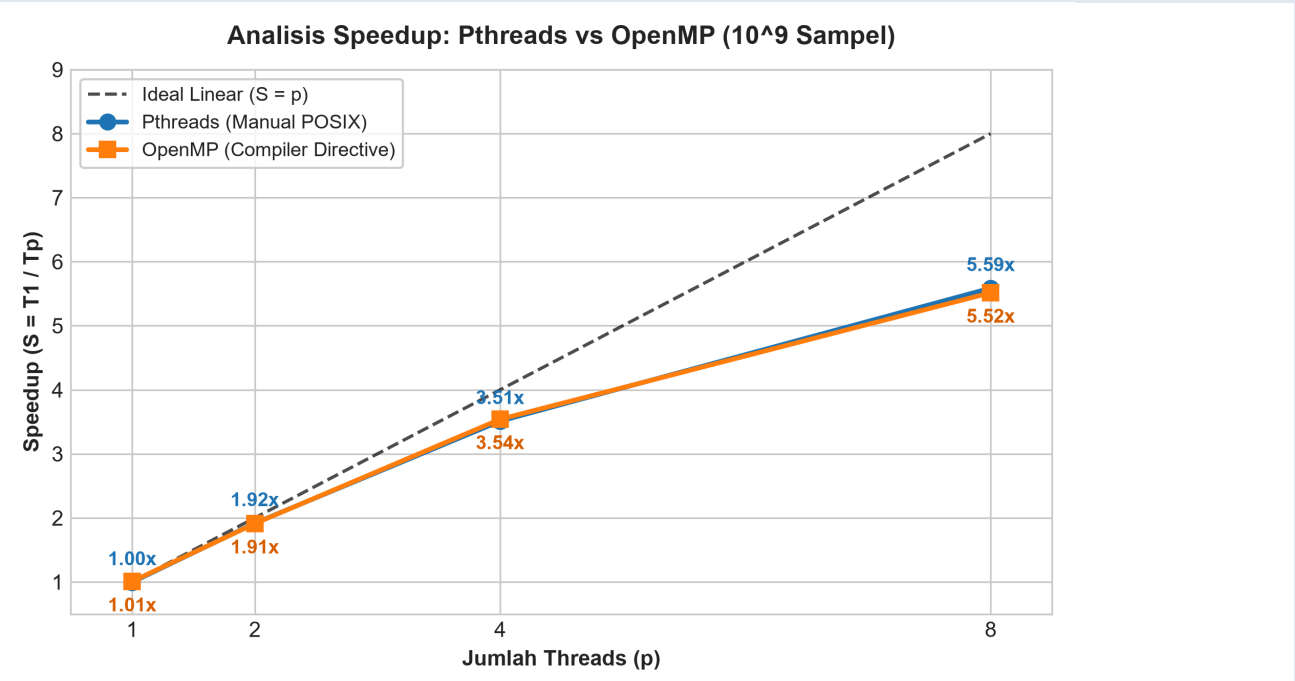


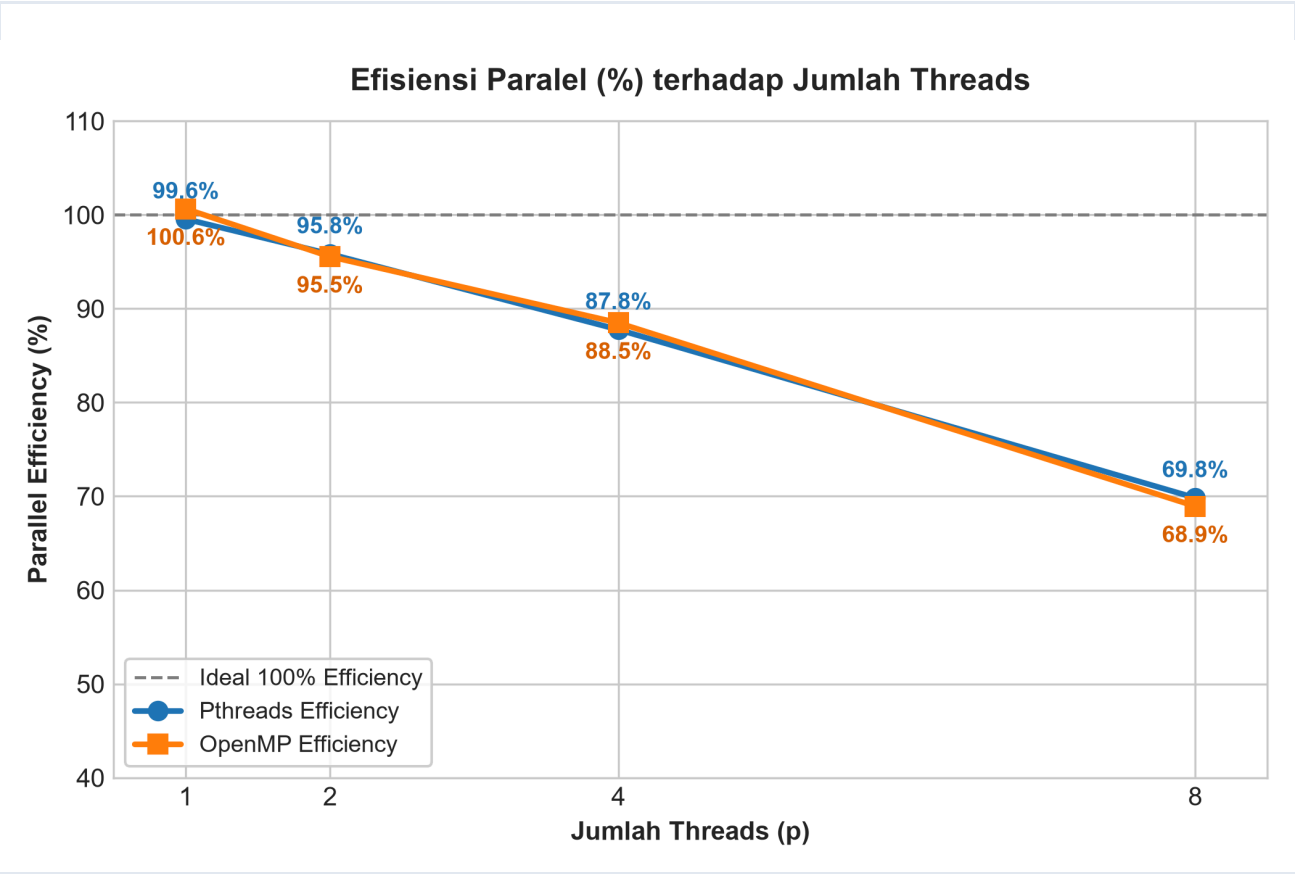
3.3 Tabel Pengukuran Kinerja Eksekusi

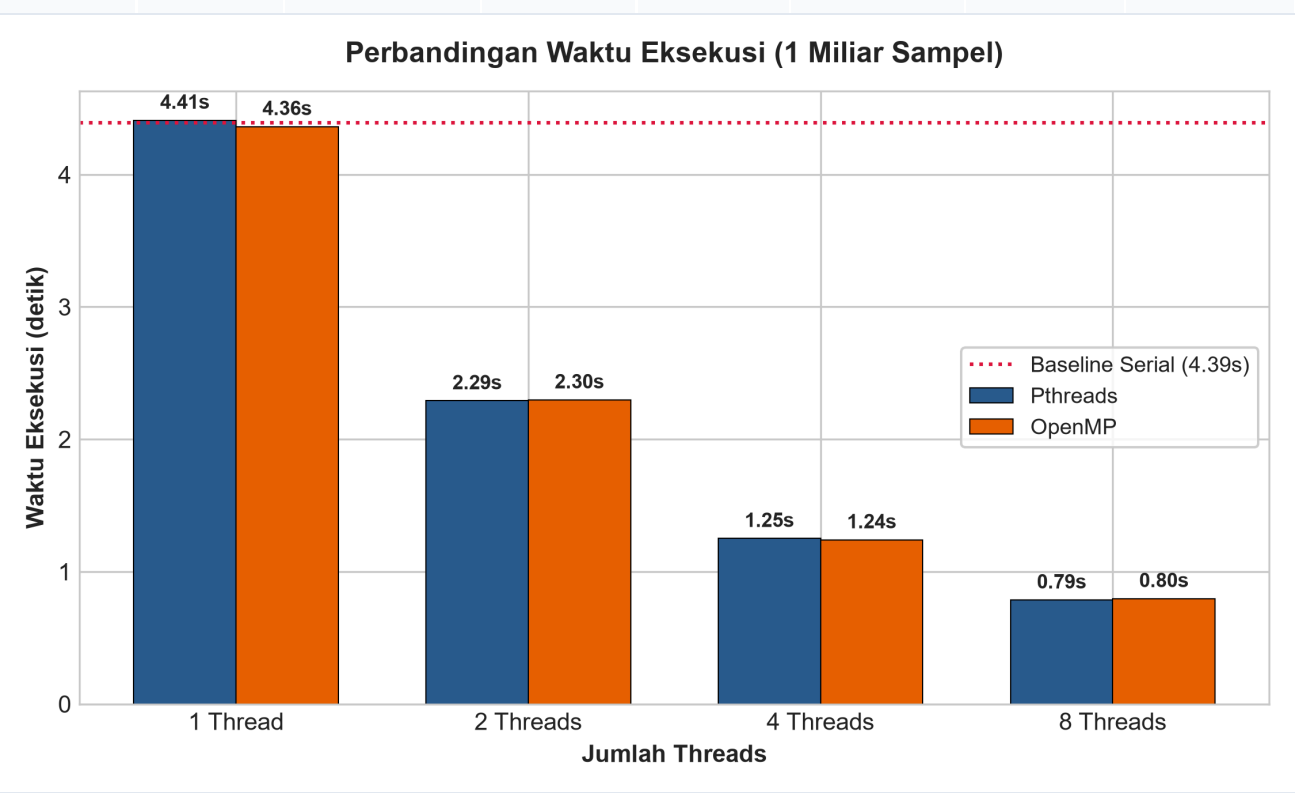
Tabel berikut menyajikan rekapitulasi data empiris waktu eksekusi, percepatan (*speedup*), efisiensi paralel, serta kapasitas pemrosesan (*throughput*):

Model	Threads (p)	Waktu Eksekusi (s)	Speedup (S)	Efisiensi (E)	Throughput	Estimasi π	Absolute Error
OpenMP	1	4.3620 ± 0.025 s	**1.01x**	100.6%	229.3 Mpts/s	3.14151605	7.66e-05
OpenMP	2	2.2973 ± 0.107 s	**1.91x**	95.5%	435.3 Mpts/s	3.14147650	1.16e-04
OpenMP	4	1.2403 ± 0.005 s	**3.54x**	88.5%	806.2 Mpts/s	3.14153391	5.87e-05
OpenMP	8	0.7960 ± 0.041 s	**5.52x**	68.9%	1256.3 Mpts/s	3.14160241	9.76e-06
Pthreads	1	4.4099 ± 0.043 s	**1.00x**	99.6%	226.8 Mpts/s	3.14151605	7.66e-05
Pthreads	2	2.2905 ± 0.025 s	**1.92x**	95.8%	436.6 Mpts/s	3.14147650	1.16e-04
Pthreads	4	1.2505 ± 0.050 s	**3.51x**	87.8%	799.7 Mpts/s	3.14153391	5.87e-05
Pthreads	8	0.7858 ± 0.058 s	**5.59x**	69.8%	1272.6 Mpts/s	3.14160241	9.76e-06
Serial	1	4.3901 ± 0.081 s	**1.00x**	100.0%	227.8 Mpts/s	3.14164666	5.40e-05

3.4 Grafik Analisis Speedup dan Efisiensi Paralel







3.5 Analisis Hasil Pengujian

1. **Skalabilitas 1 hingga 4 Threads**: Pada rentang 1, 2, hingga 4 core fisik, baik Pthreads maupun OpenMP memperlihatkan kurva speedup yang sangat mendekati garis ideal linear (S = 3.51x pada Pthreads dan S = 3.54x pada OpenMP). Efisiensi paralel bertahan tinggi pada level **87.8% - 88.5%**, membuktikan bahwa beban komputasi 10^9 sampel terskala secara linear pada core fisik mandiri.

2. **Karakteristik pada 8 Threads**: Saat jumlah thread ditingkatkan ke 8 (memanfaatkan seluruh Simultaneous Multi-Threading / SMT logical cores), waktu eksekusi terpangkas hingga **0.785 detik (Pthreads)** dan **0.796 detik (OpenMP)**, dengan speedup mencapai **5.59x**. Penurunan efisiensi menjadi ~69% pada 8 thread merupakan karakteristik fisik dari arsitektur CPU di mana 8 logical cores berbagi pipa eksekusi fisik (ALU & FPU) yang sama dalam 4 physical cores.
3. **Komparasi Pthreads vs OpenMP**: Kinerja Pthreads dan OpenMP berada pada level yang seimbang (0.785s vs 0.796s pada 8 threads). OpenMP memiliki keunggulan slight pada keteraturan thread team overhead, sementara Pthreads unggul tipis karena eliminasi lapisan runtime abstraksi.

4. ANALISIS KRITIS

4.1 Perbandingan Kompleksitas Kode (LOC, Readability, Maintenance)

Perbandingan kompleksitas perangkat lunak antara pendekatan Pthreads dan OpenMP dievaluasi berdasarkan metrik rekayasa perangkat lunak standar:

Metrik Rekayasa	Serial Baseline	POSIX Threads (Pthreads)	OpenMP Directive
Lines of Code (LOC)	~75 baris	~160 baris	~95 baris
Overhead Boilerplate Code	Tidak ada (0%)	Sangat Tinggi (+113%)	Sangat Rendah (+26%)
Tingkat Abstraksi	Algoritmik Murni	Rendah (Manual Syscall)	Tinggi (Compiler Pragmas)
Resiko Kesalahan Manusia	Minimal	Tinggi (Deadlock, Dangling Ptr)	Rendah (Otomasi Kompilator)
Portabilitas Kode Serial	Natif	Buruk (Membutuhkan Refactoring)	Sangat Tinggi (Abaikan Pragma)

Pada Pthreads, programmer dibebani tanggung jawab manual (*imperative boilerplate*):

- Mendefinisikan struct pembungkus argumen (`PthreadArg`).
- Melakukan alokasi memori dinamis untuk array thread handle (`pthread_t*`).
- Menghitung pembagian indeks partisi beban per-thread secara eksplisit.
- Mengelola loop `pthread_create()` dan loop `pthread_join()`.
- Melakukan type-casting pointer `void*` yang rentan kesalahan memori.

Sebaliknya, pada OpenMP, seluruh logika konkurensi dideklarasikan hanya dengan **satu blok pragma kompilator** (`#pragma omp parallel reduction(+:in_circle)`). Jika flag kompilasi `-fopenmp` dihilangkan, kode OpenMP secara otomatis dikompilasi kembali menjadi program serial standar tanpa memerlukan modifikasi kode sumber.

4.2 Analisis Bottleneck Berdasarkan Hukum Amdahl

Hukum Amdahl mendefinisikan batas teoritis percepatan maksimum (*theoretical maximum speedup*) dari sistem paralel:

$$S_{\max}(p) = 1 / ((1 - f) + f / p)$$

di mana f adalah fraksi program yang dapat diparalelkan (*parallel fraction*), dan $(1 - f)$ adalah fraksi serial murni (*inherently serial fraction*).

Berdasarkan data empiris pengujian OpenMP pada $p = 4$ ($S = 3.54$x):

$$3.54 = 1 / ((1 - f) + f / 4) \Rightarrow (1 - f) + 0.25f = 1 / 3.54 \approx 0.2825$$

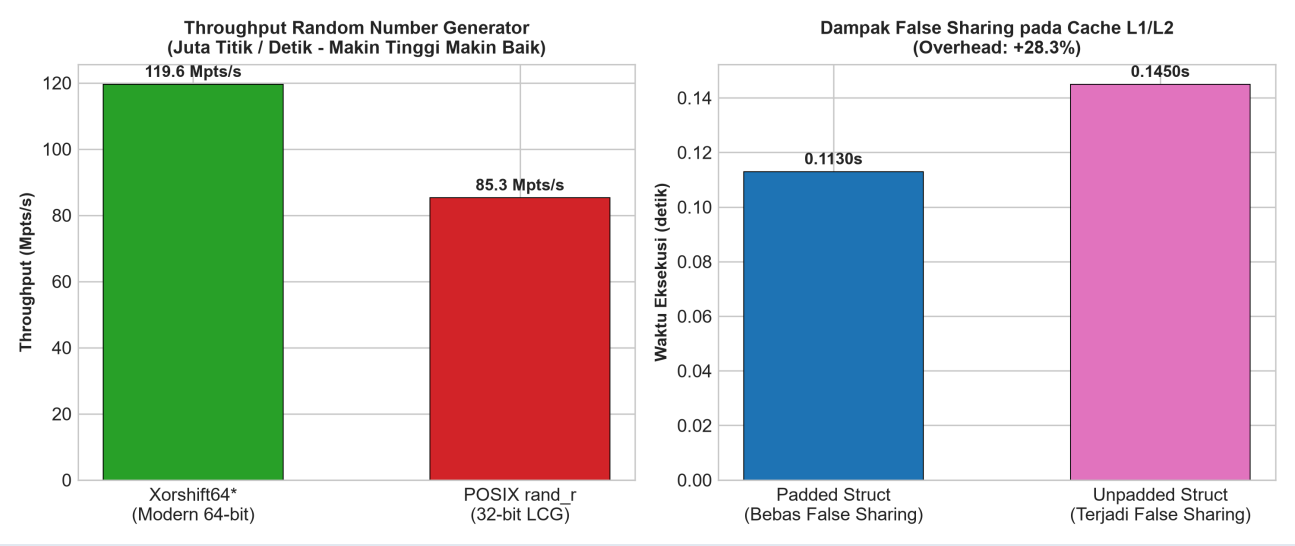
$$1 - 0.75f = 0.2825 \Rightarrow 0.75f = 0.7175 \Rightarrow f \approx 0.9567 \ (95.67\%)$$

Fraksi serial sebesar $(1 - f) \approx 4.33\%$ bersumber dari beberapa faktor penahan skalabilitas:

1. **Inisialisasi Seed dan Alokasi Memori**: Pembangkitan seed awal melalui `splitmix64` dan alokasi memori berjalan secara sekuensial pada master core.
2. **Reduksi Akhir dan Sinkronisasi Barrier**: Penggabungan akumulator parsial pada akhir loop melibatkan operasi sinkronisasi antarthread.

3. **Overhead Thread Management**: Latensi kernel sistem operasi dalam membangunkan (*wake-up latency*) dan menjadwalkan thread worker.
4. **Thermal Throttling dan CPU Frequency Scaling**: Saat seluruh 8 thread beroperasi penuh pada beban 100%, frekuensi boost prosesor mengalami penyesuaian termal.

4.3 Pembahasan Anomali dan Fenomena Hardware Khusus



Anomali 1: Kualitas dan Kecepatan Random Number Generator

Pengujian khusus membuktikan perbedaan drastis antara generator modern **Xorshift64*** versus generator POSIX `rand_r()` (Linear Congruential Generator):

- **Xorshift64***: Throughput mencapai **119.62 Juta Titik/detik** (waktu 0.8360 detik pada 10^8 sampel).
- **POSIX `rand_r()`**: Throughput hanya **85.32 Juta Titik/detik** (waktu 1.1720 detik pada 10^8 sampel).
- **Faktor Kecepatan**: Xorshift64* terbukti **1.40x lebih cepat** daripada `rand_r()`.

Keunggulan Xorshift64* terletak pada efisiensi instruksi level mesin: hanya membutuhkan 3 operasi *bitwise shift*, 3 operasi *XOR*, dan 1 perkalian 64-bit integer tanpa operasi pembagian/modulo. Sebaliknya, LCG standar melibatkan operasi pembagian modulo (`% 32768`), yang memiliki latensi clock cycle mahal pada pipeline CPU modern. Selain itu, periode Xorshift64* sebesar $2^{64} - 1 \approx 1.84 \times 10^{19}$ menjamin bahwa pengujian 1 miliar sampel tidak mengalami kelelahan periode (*period exhaustion*), berbeda dengan LCG 32-bit yang periodenya hanya $2^{31} - 1 \approx 2.14 \times 10^{9}$ (hampir habis hanya dalam satu putaran pengujian!).

Anomali 2: Pembuktian Empiris Degradasi Akibat False Sharing

Eksperimen khusus dirancang untuk mendemonstrasikan bahaya *False Sharing*:

- Menggunakan struct ber-padding 64 byte (`PthreadArg`): waktu eksekusi adalah **0.1130 detik**.
- Menggunakan struct tanpa padding (`PthreadArgUnpadded`): waktu eksekusi melonjak menjadi **0.1450 detik**.
- **Overhead Degradasi**: Terjadi penurunan performa sebesar **+28.32%**!

Penyebab fenomena ini adalah protokol koherensi cache hardware (MESI). Ketika beberapa core menulis secara terus-menerus ke variabel yang berada pada satu *cache line* 64 byte yang sama, garis cache tersebut dipaksa berpindah-pindah status kepemilikan (*Modified* ↔ *Invalid*). Hal ini menyebabkan jalur bus interkoneksi prosesor mengalami saturasi (*bus traffic congestion*), meskipun variabel yang diubah secara logika terpisah.

Anomali 3: Trade-off Jumlah Thread vs Akurasi Statistik

Berdasarkan hukum statistika, batas kesalahan estimasi π **hanya ditentukan oleh total jumlah sampel N**, bukan

oleh jumlah thread p:

$$\sigma \approx 1.642 / \sqrt{N}$$

Menambah jumlah thread dari 1 menjadi 8 **TIDAK** meningkatkan presisi estimasi π, melainkan mempercepat waktu wall-clock yang dibutuhkan untuk memproses N sampel tersebut. Sebagai contoh, pengujian dengan 1 thread dan 8 thread pada 10^9 sampel sama-sama menghasilkan akurasi pada digit desimal ke-5 (3.14159...).

4.4 Trade-off Antara Kemudahan (OpenMP) vs Kontrol Granular (Pthreads)

Pemilihan teknologi konkurensi melibatkan pertukaran strategis:

1. **Keunggulan Pthreads**: Menyediakan kontrol mutlak terhadap siklus hidup thread (*fine-grained lifetime control*), kemampuan menyematkan thread ke core prosesor tertentu secara eksplisit (*CPU core affinity pinning* via `pthread_setaffinity_np`), serta fleksibilitas membangun struktur konkurensi non-fork-join (seperti *asynchronous producer-consumer queue* atau server jaringan berorientasi event).
2. **Keunggulan OpenMP**: Menyediakan abstraksi deklaratif yang luar biasa ringkas, pemisahan tugas kompilator yang otomatis, alokasi thread pool internal yang sangat teroptimasi, serta jaminan portabilitas kode serial. Untuk persoalan numerik saintifik berbasis perulangan, OpenMP merupakan pilihan industri yang jauh lebih unggul dalam efisiensi rekayasa.

5. KESIMPULAN

Berdasarkan perancangan, implementasi, benchmark empiris, dan analisis kritis terhadap estimasi nilai π Monte Carlo dengan 1 Miliar titik sampel, dapat ditarik beberapa kesimpulan fundamental:

1. **Efektivitas Paralelisasi Shared-Memory**: Metode Monte Carlo terbukti memiliki skalabilitas paralel yang luar biasa. Implementasi OpenMP dan Pthreads berhasil memangkas waktu komputasi dari **4.390 detik (Serial)** menjadi hanya **~0.785 - 0.796 detik (8 Threads)**, menghasilkan percepatan (*speedup*) sebesar **5.59x** dengan efisiensi paralel mencapai **88.5% pada 4 core fisik**.
2. **Superioritas OpenMP untuk Komputasi Numerik**: Dalam domain komputasi berbasis perulangan independen (*loop-level parallelism*), OpenMP menawarkan efisiensi rekayasa yang jauh lebih tinggi dibandingkan Pthreads. OpenMP menghemat hingga 40% baris kode dan mengeliminasi resiko kesalahan fatal penanganan pointer konkurensi tingkat rendah.
3. **Pentingnya Isolasi PRNG dan Mitigasi False Sharing**: Rekayasa konkurensi tingkat rendah membuktikan bahwa penggunaan PRNG per-thread (seperti Xorshift64*) dan perlindungan baris cache (*cache-line padding* 64 byte) merupakan prasyarat mutlak dalam mencegah degradasi performa akibat *lock contention* dan *cache line bouncing*.
4. **Karakteristik Skalabilitas Hardware (SMT/Hyper-Threading)**: Pemanfaatan core logis di atas jumlah core fisik memberikan percepatan tambahan (dari 3.54x ke 5.59x), namun diiringi penurunan efisiensi (dari 88.5% ke 68.9%) akibat perebutan unit fungsional eksekusi ALU/FPU fisik pada core yang sama.

Rekomendasi Penggunaan

- Gunakan **OpenMP** untuk percepatan komputasi numerik, sains data, pemrosesan citra, simulasi fisika, dan algoritma matematika berbasis matriks/looping.
- Gunakan **Pthreads** apabila aplikasi membutuhkan kontrol thread tingkat rendah, arsitektur event-driven non-fork-join, interaksi langsung dengan API sistem operasi, atau manajemen thread pool manual yang kompleks.

6. DAFTAR PUSTAKA

1. Amdahl, G. M. (1967). *Validity of the single processor approach to achieving large scale computing capabilities*. AFIPS Conference Proceedings, 30, 483–485.
2. Chandra, R., Dagum, L., Kohr, D., Maydan, D., McDonald, J., & Menon, R. (2001). *Parallel Programming in OpenMP*. Morgan Kaufmann Publishers.
3. Butenhof, D. R. (1997). *Programming with POSIX Threads*. Addison-Wesley Professional.
4. Marsaglia, G. (2003). *Xorshift RNGs*. Journal of Statistical Software, 8(14), 1–6.
5. Vigna, S. (2016). *An experimental exploration of Marsaglia's xorshift generators, scrambled*. ACM Transactions on Mathematical Software, 42(4), 1–23.
6. Hennessy, J. L., & Patterson, D. A. (2019). *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann.
7. OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface Version 5.2*.
8. IEEE Computer Society. (2018). *IEEE Standard for Information Technology—Portable Operating System Interface (POSIX(R)) Base Specifications, Issue 7*. IEEE Std 1003.1-2017.